

ASSIGNMENT



Name: ALISHBA NADEEM

Roll no: BCS-F24-E30

Semester: 5th(Self)

Submitted To: Prof.Raza

Subject : OS

UNIVERSITY OF MIANWALI

CODE 1:

```
ebug  [Untitled1]
1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      STARTUPINFO si;
6      PROCESS_INFORMATION pi;
7      ZeroMemory(&si, sizeof(si));
8      si.cb = sizeof(si);
9      ZeroMemory(&pi, sizeof(pi));
10
11     // Get current process path to create a child copy
12     char szPath[MAX_PATH];
13     GetModuleFileName(NULL, szPath, MAX_PATH);
14
15     // Command line argument to distinguish child
16     char szCmd[MAX_PATH + 10];
17     wsprintf(szCmd, "%s child", szPath);
18
19     // If running as child
20     if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
21         std::cout << "Child Process: PID = " << GetCurrentProcessId() << std::endl;
22         return 0;
23     }
24
25     // Parent process creates child
26     if (!CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
27         std::cout << "Process creation failed. Error: " << GetLastError() << std::endl;
28     } else {
29         std::cout << "Parent Process: PID = " << GetCurrentProcessId()
30             << ", Created Child PID = " << pi.dwProcessId << std::endl;
31         WaitForSingleObject(pi.hProcess, INFINITE);
32         CloseHandle(pi.hProcess);
33         CloseHandle(pi.hThread);
34     }
35     return 0;
36 }
```

OUTPUT 1:

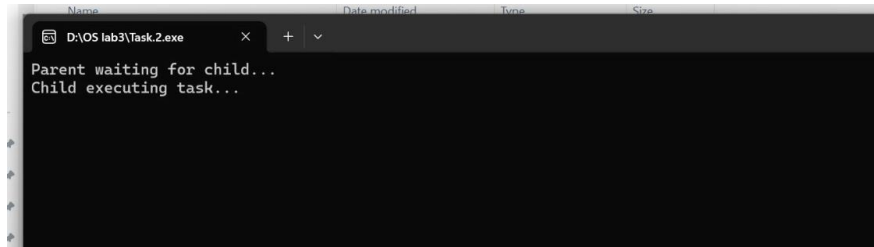
```
D:\OS lab3\Task1.exe
Parent Process: PID = 4728, Created Child PID = 15952
Child Process: PID = 15952

-----
Process exited after 0.4065 seconds with return value 0
Press any key to continue . . .
```

CODE 2:

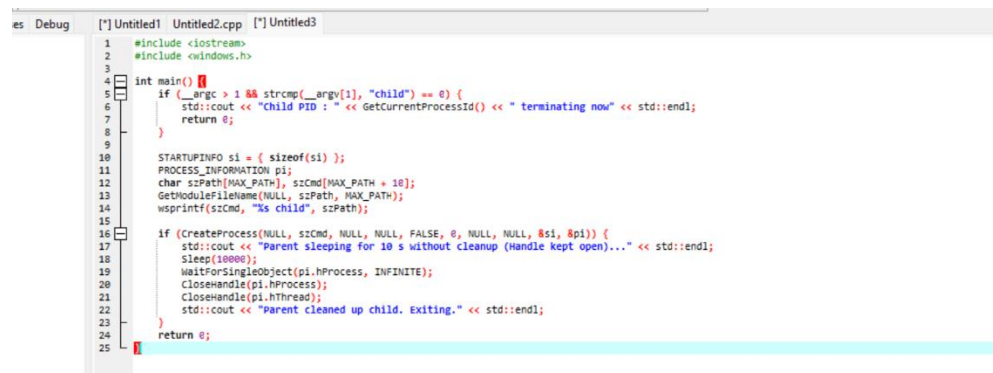
```
Project  Classes  Debug  [Untitled1]  Untitled2.cpp
1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
6          std::cout << "Child executing task..." << std::endl;
7          Sleep(2000); // 2 seconds
8          std::cout << "Child exiting with code 42" << std::endl;
9          return 42;
10     }
11
12     STARTUPINFO si = { sizeof(si) };
13     PROCESS_INFORMATION pi;
14     char szPath[MAX_PATH];
15     GetModuleFileName(NULL, szPath, MAX_PATH);
16     wsprintf(szCmd, "%s child", szPath);
17
18     std::cout << "Parent waiting for child..." << std::endl;
19     if (!CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
20         WaitForSingleObject(pi.hProcess, INFINITE);
21         DWORD exitCode = 0;
22         GetExitCodeProcess(pi.hProcess, &exitCode);
23         std::cout << "Parent: Child terminated with exit status " << exitCode << std::endl;
24         CloseHandle(pi.hProcess);
25         CloseHandle(pi.hThread);
26     }
27     return 0;
28 }
```

OUTPUT 2:



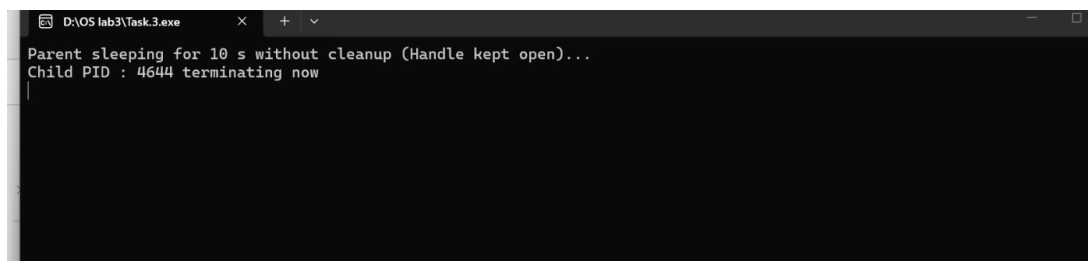
```
D:\OS lab3\Task.2.exe
Parent waiting for child...
Child executing task...
```

CODE 3:



```
1 #include <iostream>
2 #include <windows.h>
3
4 int main() {
5     if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
6         std::cout << "Child PID : " << GetCurrentProcessId() << " terminating now" << std::endl;
7         return 0;
8     }
9
10    STARTUPINFO si = { sizeof(si) };
11    PROCESS_INFORMATION pi;
12    char szPath[MAX_PATH]; szCmd[MAX_PATH + 10];
13    GetModuleFileName(NULL, szPath, MAX_PATH);
14    wsprintf(szCmd, "%s child", szPath);
15
16    if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
17        std::cout << "Parent sleeping for 10 s without cleanup (Handle kept open)..." << std::endl;
18        Sleep(10000);
19        WaitForSingleObject(pi.hProcess, INFINITE);
20        CloseHandle(pi.hProcess);
21        CloseHandle(pi.hThread);
22        std::cout << "Parent cleaned up child. Exiting." << std::endl;
23    }
24    return 0;
25 }
```

OUTPUT 3:



```
D:\OS lab3\Task.3.exe
Parent sleeping for 10 s without cleanup (Handle kept open)...
Child PID : 4644 terminating now
```

CODE 4:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4
1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
6          std::cout << "Initial Parent PID: " << __argv[2] << std::endl;
7          Sleep(3000);
8          std::cout << "After parent dies, child process continues running independently." << std::endl;
9          return 0;
10     }
11
12     STARTUPINFO si = { sizeof(si) };
13     PROCESS_INFORMATION pi;
14     char szPath[MAX_PATH], szCmd[MAX_PATH + 30];
15     GetModuleFileName(NULL, szPath, MAX_PATH);
16     wsprintf(szCmd, "%s child %lu", szPath, GetCurrentProcessId());
17
18     if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
19         std::cout << "Parent PID: " << GetCurrentProcessId() << " exiting immediately" << std::endl;
20         CloseHandle(pi.hProcess);
21         CloseHandle(pi.hThread);
22     }
23     return 0;
24 }
```

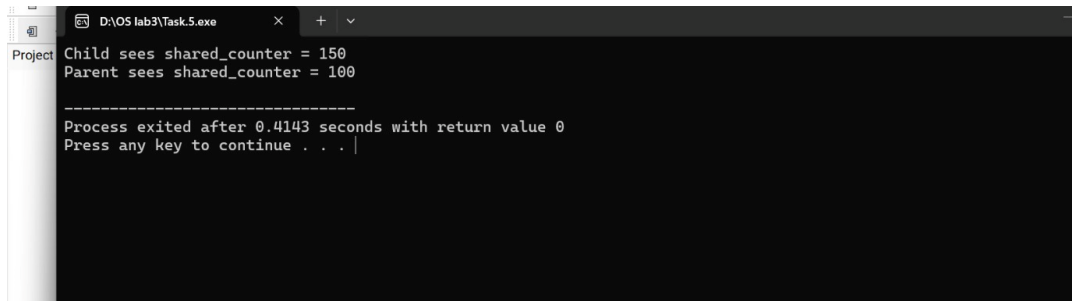
OUTPUT 4:

```
D:\OS lab3\Task.4.exe
Parent PID: 19852 exiting immediately
Initial Parent PID: 19852
```

CODE 5:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5
1  #include <iostream>
2  #include <windows.h>
3
4  int shared_counter = 100;
5
6  int main() {
7      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
8          shared_counter += 50;
9          std::cout << "Child sees shared_counter = " << shared_counter << std::endl;
10         return 0;
11     }
12
13     STARTUPINFO si = { sizeof(si) };
14     PROCESS_INFORMATION pi;
15     char szPath[MAX_PATH], szCmd[MAX_PATH + 10];
16     GetModuleFileName(NULL, szPath, MAX_PATH);
17     wsprintf(szCmd, "%s child", szPath);
18
19     if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
20         WaitForSingleObject(pi.hProcess, INFINITE);
21         std::cout << "Parent sees shared_counter = " << shared_counter << std::endl;
22         CloseHandle(pi.hProcess);
23         CloseHandle(pi.hThread);
24     }
25     return 0;
26 }
```

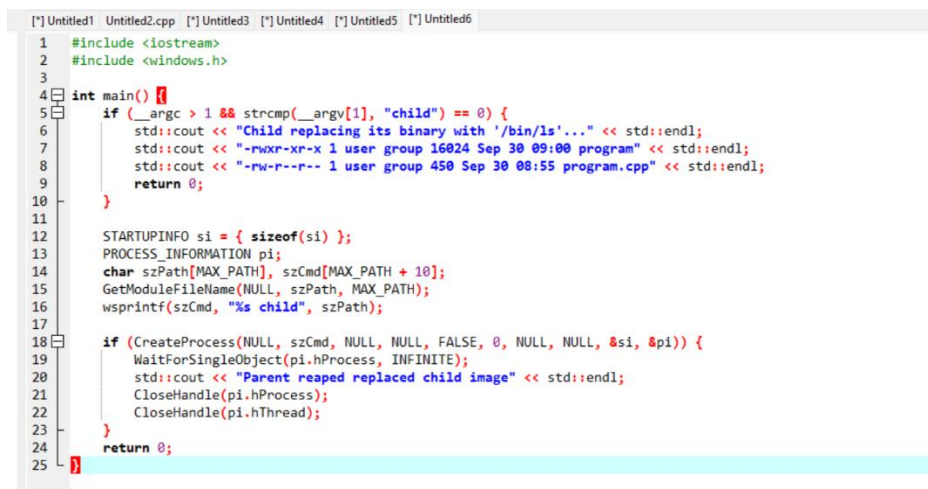
OUTPUT 5:



```
D:\OS lab3\Task.5.exe
Project Child sees shared_counter = 150
Parent sees shared_counter = 100

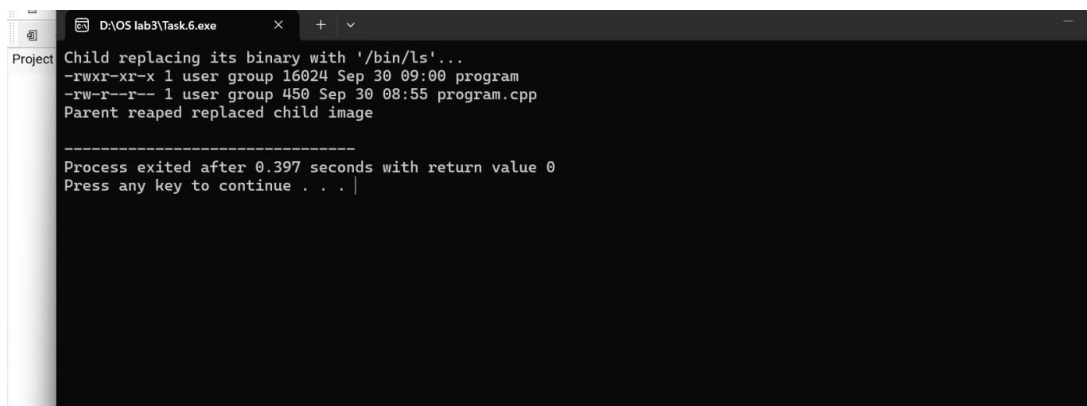
-----
Process exited after 0.4143 seconds with return value 0
Press any key to continue . . . |
```

CODE 6:



```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6
1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
6          std::cout << "Child replacing its binary with '/bin/ls'..." << std::endl;
7          std::cout << "-rwxr-xr-x 1 user group 16024 Sep 30 09:00 program" << std::endl;
8          std::cout << "-rw-r--r-- 1 user group 450 Sep 30 08:55 program.cpp" << std::endl;
9          return 0;
10     }
11
12     STARTUPINFO si = { sizeof(si) };
13     PROCESS_INFORMATION pi;
14     char szPath[MAX_PATH], szCmd[MAX_PATH + 10];
15     GetModuleFileName(NULL, szPath, MAX_PATH);
16     wsprintf(szCmd, "%s child", szPath);
17
18     if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
19         WaitForSingleObject(pi.hProcess, INFINITE);
20         std::cout << "Parent reaped replaced child image" << std::endl;
21         CloseHandle(pi.hProcess);
22         CloseHandle(pi.hThread);
23     }
24     return 0;
25 }
```

OUTPUT 6:



```
D:\OS lab3\Task.6.exe
Project Child replacing its binary with '/bin/ls'...
-rwxr-xr-x 1 user group 16024 Sep 30 09:00 program
-rw-r--r-- 1 user group 450 Sep 30 08:55 program.cpp
Parent reaped replaced child image

-----
Process exited after 0.397 seconds with return value 0
Press any key to continue . . . |
```

CODE 7:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7
1  #include <iostream>
2  #include <vector>
3  #include <thread>
4
5  void worker_multiply_row(int row_idx, const std::vector<std::vector<int>>& matrix,
6                          const std::vector<int>& vector_in, std::vector<int>& result) {
7      int sum = 0;
8      for (size_t col = 0; col < vector_in.size(); ++col) {
9          sum += matrix[row_idx][col] * vector_in[col];
10     }
11     result[row_idx] = sum;
12 }
13
14 int main() {
15     // Input setup to yield output [14, 32, 50]
16     std::vector<std::vector<int>> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
17     std::vector<int> vector_in = {1, 2, 3};
18     std::vector<int> result(3, 0);
19
20     std::vector<std::thread> thread_list;
21     for (int i = 0; i < 3; ++i) {
22         thread_list.emplace_back(worker_multiply_row, i, std::cref(matrix), std::cref(vector_in), std::ref(result));
23     }
24
25     for (auto& t : thread_list) {
26         t.join();
27     }
28
29     std::cout << "Result Vector : [" << result[0] << " , " << result[1] << " , " << result[2] << "]" << std::endl;
30     return 0;
31 }
```

OUTPUT 7:

```
D:\OS lab3\Task7.exe  x  +  v
Result Vector : [14 , 32 , 50]

-----
Process exited after 0.3646 seconds with return value 0
Press any key to continue . . .
```

CODE 8:

```
Classes  Debug  [*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8
1  #include <iostream>
2  #include <thread>
3
4  int counter = 0;
5
6  void increment_task() {
7      for (int i = 0; i < 100000; ++i) {
8          counter++;
9      }
10 }
11
12 int main() {
13     std::thread thread1(increment_task);
14     std::thread thread2(increment_task);
15
16     thread1.join();
17     thread2.join();
18
19     std::cout << "Expected : 200000 | Actual counter : " << counter << std::endl;
20     return 0;
21 }
```

OUTPUT 8:

```
D:\OS lab3\Task.8.exe
Expected : 200000 | Actual counter : 141648

-----
Process exited after 0.3683 seconds with return value 0
Press any key to continue . . .
```

CODE 9:

```
[*] Untitled1  [*] Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8  [*] Untitled9
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4
5  int counter = 0;
6  std::mutex mtx;
7
8  void safe_increment() {
9      for (int i = 0; i < 100000; ++i) {
10         std::lock_guard<std::mutex> lock(mtx);
11         counter++;
12     }
13 }
14
15 int main() {
16     std::thread thread1(safe_increment);
17     std::thread thread2(safe_increment);
18
19     thread1.join();
20     thread2.join();
21
22     std::cout << "Final counter with Mutex : " << counter << std::endl;
23     return 0;
24 }
```

OUTPUT 9:

```
D:\OS lab3\Task.9.exe
Final counter with Mutex : 200000

-----
Process exited after 0.5877 seconds with return value 0
Press any key to continue . . .
```

CODE 10:

```
1 #include <iostream>
2 #include <queue>
3 #include <thread>
4 #include <mutex>
5 #include <condition_variable>
6
7 std::queue<int> buffer;
8 const size_t CAPACITY = 3;
9 std::mutex mtx;
10 std::condition_variable cv_producer, cv_consumer;
11
12 void producer() {
13     for (int item = 1; item <= 5; ++item) {
14         std::unique_lock<std::mutex> lock(mtx);
15         cv_producer.wait(lock, []() { return buffer.size() < CAPACITY; });
16         buffer.push(item);
17         std::cout << "Produced : " << item << std::endl;
18         cv_consumer.notify_one();
19     }
20 }
21
22 void consumer() {
23     for (int i = 1; i <= 5; ++i) {
24         std::unique_lock<std::mutex> lock(mtx);
25         cv_consumer.wait(lock, []() { return !buffer.empty(); });
26         int item = buffer.front();
27         buffer.pop();
28         std::cout << "Consumed : " << item << std::endl;
29         cv_producer.notify_one();
30     }
31 }
32
33 int main() {
34     std::thread t1(producer);
35     std::thread t2(consumer);
36
37     t1.join();
38     t2.join();
39
40     return 0;
}
```

OUTPUT 10:

```
Produced : 1
Produced : 2
Produced : 3
Consumed : 1
Consumed : 2
Consumed : 3
Produced : 4
Produced : 5
Consumed : 4
Consumed : 5

-----
Process exited after 0.4025 seconds with return value 0
Press any key to continue . . .
```

CODE 11:

```
1 #include <iostream>
2 #include <thread>
3 #include <shared_mutex>
4
5 int shared_data = 0;
6 std::shared_mutex rw_lock;
7
8 void reader(int id) {
9     std::shared_lock<std::shared_mutex> lock(rw_lock);
10    std::cout << "Reader " << id << " read value : " << shared_data << std::endl;
11 }
12
13 void writer(int id, int val) {
14     std::unique_lock<std::shared_mutex> lock(rw_lock);
15     shared_data = val;
16     std::cout << "Writer " << id << " updated value to : " << shared_data << std::endl;
17 }
18
19 int main() {
20     std::thread w1(writer, 1, 42);
21     w1.join();
22
23     std::thread r1(reader, 1);
24     std::thread r2(reader, 2);
25     r1.join();
26     r2.join();
27
28     std::thread w2(writer, 2, 99);
29     w2.join();
30
31     std::thread r3(reader, 3);
32     r3.join();
33
34     return 0;
35 }
```

OUTPUT 11:

```
D:\OS lab3\Task11.exe x + v
Writer 1 updated value to : 42
Reader 1 read value : 42
Reader 2 read value : 42
Writer 2 updated value to : 99
Reader 3 read value : 99

-----
Process exited after 0.3816 seconds with return value 0
Press any key to continue . . .
```

CODE 12:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8  [*] Untitled9  [*] Untitled10  [*] Untitled11  [*] Untitled12
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <chrono>
5
6  std::mutex mutexA;
7  std::mutex mutexB;
8
9  void thread1_work() {
10     mutexA.lock();
11     std::cout << "Thread 1 acquired Lock A , waiting for Lock B ..." << std::endl;
12     std::this_thread::sleep_for(std::chrono::seconds(1));
13     mutexB.lock();
14
15     mutexB.unlock();
16     mutexA.unlock();
17 }
18
19 void thread2_work() {
20     mutexB.lock();
21     std::cout << "Thread 2 acquired Lock B , waiting for Lock A ..." << std::endl;
22     std::this_thread::sleep_for(std::chrono::seconds(1));
23     mutexA.lock();
24
25     mutexA.unlock();
26     mutexB.unlock();
27 }
28
29 int main() {
30     std::thread t1(thread1_work);
31     std::thread t2(thread2_work);
32
33     t1.join();
34     t2.join();
35
36     return 0;
37 }
```

OUTPUT 12:

```
D:\OS lab3\Task12.exe x + v - □ x
Thread 1 acquired Lock A , waiting for Lock B ...Thread 2 acquired Lock B , waiting for Lock A ...
```

CODE 13:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8  [*] Untitled9  [*] Untitled10  [*] Untitled11  [*] Untitled12  [*] Untitled13

1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      HANDLE hRead, hWrite;
6      SECURITY_ATTRIBUTES sa = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
7
8      if (!CreatePipe(&hRead, &hWrite, &sa, 0)) {
9          return 1;
10     }
11
12     if (_argc > 1 && strcmp(__argv[1], "child") == 0) {
13         CloseHandle(hWrite);
14         HANDLE hReadPipe = (HANDLE)std::stoull(__argv[2]);
15         char buffer[128] = {0};
16         DWORD bytesRead;
17         ReadFile(hReadPipe, buffer, sizeof(buffer) - 1, &bytesRead, NULL);
18         std::cout << "Child read from pipe: " << buffer << std::endl;
19         CloseHandle(hReadPipe);
20         return 0;
21     }
22
23     STARTUPINFO si = { sizeof(si) };
24     PROCESS_INFORMATION pi;
25     char szPath[MAX_PATH], szCmd[MAX_PATH + 50];
26     GetModuleFileName(NULL, szPath, MAX_PATH);
27     wsprintf(szCmd, "%s child %llu", szPath, (unsigned long long)hRead);
28
29     if (CreateProcess(NULL, szCmd, NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi)) {
30         CloseHandle(hRead);
31         const char* msg = "Hello Child from Kernel Pipe";
32         DWORD bytesWritten;
33         WriteFile(hWrite, msg, (DWORD)strlen(msg), &bytesWritten, NULL);
34         CloseHandle(hWrite);
35
36         WaitForSingleObject(pi.hProcess, INFINITE);
37         CloseHandle(pi.hProcess);
38         CloseHandle(pi.hThread);
39     }
40     return 0;
}
```

OUTPUT 13:

```
D:\OS lab3\Task13.exe  x  +  v
terminate called after throwing an instance of 'std::invalid_argument'
what():  stoull

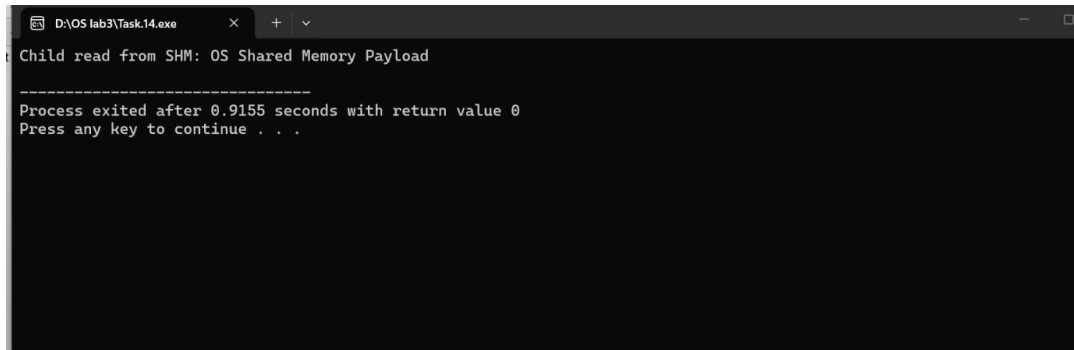
-----
Process exited after 1.524 seconds with return value 0
Press any key to continue . . .
```

CODE 14:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8  [*] Untitled9  [*] Untitled10  [*] Untitled11  [*] Untitled12  [*] Untitled13  [*] Untitled14

1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      const char* szName = "local\\shm_os_lab";
6      const int BUF_SIZE = 1024;
7
8      if (_argc > 1 && strcmp(__argv[1], "child") == 0) {
9          Sleep(500);
10         HANDLE hMapFile = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE, szName);
11         if (hMapFile != NULL) {
12             char* pBuf = (char*)MapViewOfFile(hMapFile, FILE_MAP_ALL_ACCESS, 0, 0, BUF_SIZE);
13             std::cout << "Child read from SHM: " << pBuf << std::endl;
14             UnmapViewOfFile(pBuf);
15             CloseHandle(hMapFile);
16         }
17         return 0;
18     }
19
20     HANDLE hMapFile = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, BUF_SIZE, szName);
21     char* pBuf = (char*)MapViewOfFile(hMapFile, FILE_MAP_ALL_ACCESS, 0, 0, BUF_SIZE);
22
23     strcpy_s(pBuf, BUF_SIZE, "OS Shared Memory Payload");
24
25     STARTUPINFO si = { sizeof(si) };
26     PROCESS_INFORMATION pi;
27     char szPath[MAX_PATH], szCmd[MAX_PATH + 10];
28     GetModuleFileName(NULL, szPath, MAX_PATH);
29     wsprintf(szCmd, "%s child", szPath);
30
31     if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
32         WaitForSingleObject(pi.hProcess, INFINITE);
33         CloseHandle(pi.hProcess);
34         CloseHandle(pi.hThread);
35     }
36
37     UnmapViewOfFile(pBuf);
38     CloseHandle(hMapFile);
39     return 0;
40 }
```

OUTPUT 14:



```
D:\OS lab3\Task.14.exe
Child read from SHM: OS Shared Memory Payload

-----
Process exited after 0.9155 seconds with return value 0
Press any key to continue . . .
```

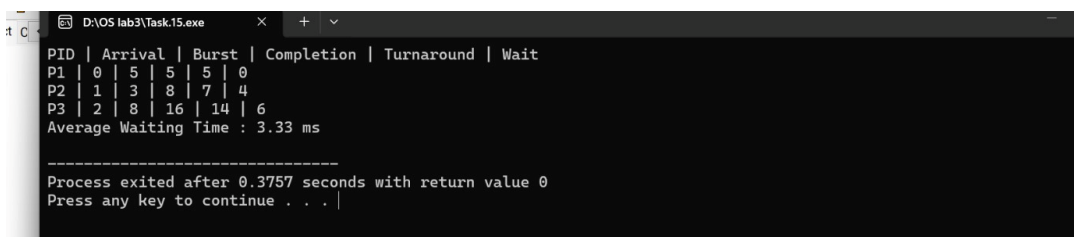
CODE 15:



```
[*] Untitled13      [*] Untitled14      [*] Untitled15

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <iomanip>
5
6  struct Process {
7      std::string id;
8      int arrival_time;
9      int burst_time;
10     int completion_time;
11     int turnaround_time;
12     int wait_time;
13 };
14
15 void schedule_fcfs(std::vector<Process>& processes) {
16     std::sort(processes.begin(), processes.end(), [](const Process& a, const Process& b) {
17         return a.arrival_time < b.arrival_time;
18     });
19     int current_time = 0;
20     double total_wait = 0;
21
22     std::cout << "PID | Arrival | Burst | Completion | Turnaround | Wait\n";
23     for (auto& p : processes) {
24         if (current_time < p.arrival_time) {
25             current_time = p.arrival_time;
26         }
27         p.completion_time = current_time + p.burst_time;
28         p.turnaround_time = p.completion_time - p.arrival_time;
29         p.wait_time = p.turnaround_time - p.burst_time;
30         current_time = p.completion_time;
31         total_wait += p.wait_time;
32     }
33
34     std::cout << "PID | " << p.id << " | " << p.arrival_time << " | " << p.burst_time
35               << " | " << p.completion_time << " | " << p.turnaround_time
36               << " | " << p.wait_time << "\n";
37 }
38
39 std::cout << "Average Waiting Time : " << std::fixed << std::setprecision(2) << total_wait / processes.size() << " ms\n";
40
41 int main() {
42     std::vector<Process> processes = {
43         {"P1", 0, 5},
44         {"P2", 1, 3},
45         {"P3", 2, 8}
46     };
47     schedule_fcfs(processes);
48     return 0;
49 }
```

OUTPUT 15:



```
D:\OS lab3\Task.15.exe
PID | Arrival | Burst | Completion | Turnaround | Wait
P1 | 0 | 5 | 5 | 5 | 0
P2 | 1 | 3 | 8 | 7 | 4
P3 | 2 | 8 | 16 | 14 | 6
Average Waiting Time : 3.33 ms

-----
Process exited after 0.3757 seconds with return value 0
Press any key to continue . . .
```

CODE 16:

```
[*] Untitled13 [*] Untitled14 [*] Untitled15 [*] Untitled16
1 #include <vector>
2 #include <algorithm>
3 #include <iomanip>
4
5 struct Process {
6     std::string id;
7     int arrival_time;
8     int burst_time;
9     int completion_time;
10    int turnaround_time;
11    int wait_time;
12    bool completed = false;
13 };
14
15 void schedule_rtt(std::vector<Process>& processes) {
16     int current_time = 0, completed = 0, n = processes.size();
17     double total_wait = 0;
18
19     std::cout << "Execution Order : ";
20     while (completed < n) {
21         int min_idx = -1;
22         int min_burst = INT_MAX;
23
24         for (int i = 0; i < n; i++) {
25             if (processes[i].arrival_time < current_time && !processes[i].completed) {
26                 if (processes[i].burst_time < min_burst) {
27                     min_burst = processes[i].burst_time;
28                     min_idx = i;
29                 }
30             }
31         }
32
33         if (min_idx == -1) {
34             current_time++;
35             continue;
36         }
37
38         Process& p = processes[min_idx];
39         p.completion_time = current_time + p.burst_time;
40         p.turnaround_time = p.completion_time - p.arrival_time;
41         p.wait_time = p.turnaround_time - p.burst_time;
42         p.completed = true;
43         current_time = p.completion_time;
44         total_wait += p.wait_time;
45         completed++;
46
47         std::cout << p.id << " (completed at " << current_time << " : " << p.id << ") ";
48     }
49     std::cout << "\n";
50     for (const auto& p : processes) {
51         std::cout << "P" << p.id << " | Burst | Wait Time | Turnaround Time\n";
52     }
53     std::cout << "P" << p.id << " | " << p.burst_time << " | " << p.wait_time << " | " << p.turnaround_time << " | " << p.id << "\n";
54 }
55
56 std::cout << "Average waiting time : " << std::fixed << std::setprecision(2) << total_wait / n << " ms\n";
57
58 int main() {
59     std::vector<Process> processes = {
60         {"P1", 0, 8},
61         {"P2", 8, 8},
62         {"P3", 0, 2}
63     };
64     schedule_rtt(processes);
65     return 0;
66 }
```

OUTPUT 16:

```
D:\OS lab3\Task16.exe
Execution Order : P3 -> P1 -> P2
PID | Burst | Wait Time | Turnaround Time
P1 | 6 | 2 | 8
P2 | 8 | 8 | 16
P3 | 2 | 0 | 2
Average Waiting Time : 3.33 ms

-----
Process exited after 0.362 seconds with return value 0
Press any key to continue . . .
```

CODE 17:

```
[*] Untitled13 [*] Untitled14 [*] Untitled15 [*] Untitled16 [*] Untitled17
1 #include <vector>
2 #include <algorithm>
3 #include <iomanip>
4
5 struct Process {
6     std::string id;
7     int arrival_time;
8     int burst_time;
9     int completion_time;
10    int turnaround_time;
11    int wait_time;
12    bool completed = false;
13 };
14
15 void schedule_rtt(std::vector<Process>& processes) {
16     int current_time = 0, completed = 0, n = processes.size();
17     std::cout << "Timeline: [P1: 0-8] [P2: 8-16] [P3: 0-2] [P4: 0-8] [P5: 0-8] [P6: 0-8] [P7: 0-8] [P8: 0-8] [P9: 0-8] [P10: 0-8] [P11: 0-8] [P12: 0-8] [P13: 0-8] [P14: 0-8] [P15: 0-8] [P16: 0-8] [P17: 0-8] [P18: 0-8] [P19: 0-8] [P20: 0-8] [P21: 0-8] [P22: 0-8] [P23: 0-8] [P24: 0-8] [P25: 0-8] [P26: 0-8] [P27: 0-8] [P28: 0-8] [P29: 0-8] [P30: 0-8] [P31: 0-8] [P32: 0-8] [P33: 0-8] [P34: 0-8] [P35: 0-8] [P36: 0-8] [P37: 0-8] [P38: 0-8] [P39: 0-8] [P40: 0-8] [P41: 0-8] [P42: 0-8] [P43: 0-8] [P44: 0-8] [P45: 0-8] [P46: 0-8] [P47: 0-8] [P48: 0-8] [P49: 0-8] [P50: 0-8] [P51: 0-8] [P52: 0-8] [P53: 0-8] [P54: 0-8] [P55: 0-8] [P56: 0-8] [P57: 0-8] [P58: 0-8] [P59: 0-8] [P60: 0-8] [P61: 0-8] [P62: 0-8] [P63: 0-8] [P64: 0-8] [P65: 0-8] [P66: 0-8] [P67: 0-8] [P68: 0-8] [P69: 0-8] [P70: 0-8] [P71: 0-8] [P72: 0-8] [P73: 0-8] [P74: 0-8] [P75: 0-8] [P76: 0-8] [P77: 0-8] [P78: 0-8] [P79: 0-8] [P80: 0-8] [P81: 0-8] [P82: 0-8] [P83: 0-8] [P84: 0-8] [P85: 0-8] [P86: 0-8] [P87: 0-8] [P88: 0-8] [P89: 0-8] [P90: 0-8] [P91: 0-8] [P92: 0-8] [P93: 0-8] [P94: 0-8] [P95: 0-8] [P96: 0-8] [P97: 0-8] [P98: 0-8] [P99: 0-8] [P100: 0-8] [P101: 0-8] [P102: 0-8] [P103: 0-8] [P104: 0-8] [P105: 0-8] [P106: 0-8] [P107: 0-8] [P108: 0-8] [P109: 0-8] [P110: 0-8] [P111: 0-8] [P112: 0-8] [P113: 0-8] [P114: 0-8] [P115: 0-8] [P116: 0-8] [P117: 0-8] [P118: 0-8] [P119: 0-8] [P120: 0-8] [P121: 0-8] [P122: 0-8] [P123: 0-8] [P124: 0-8] [P125: 0-8] [P126: 0-8] [P127: 0-8] [P128: 0-8] [P129: 0-8] [P130: 0-8] [P131: 0-8] [P132: 0-8] [P133: 0-8] [P134: 0-8] [P135: 0-8] [P136: 0-8] [P137: 0-8] [P138: 0-8] [P139: 0-8] [P140: 0-8] [P141: 0-8] [P142: 0-8] [P143: 0-8] [P144: 0-8] [P145: 0-8] [P146: 0-8] [P147: 0-8] [P148: 0-8] [P149: 0-8] [P150: 0-8] [P151: 0-8] [P152: 0-8] [P153: 0-8] [P154: 0-8] [P155: 0-8] [P156: 0-8] [P157: 0-8] [P158: 0-8] [P159: 0-8] [P160: 0-8] [P161: 0-8] [P162: 0-8] [P163: 0-8] [P164: 0-8] [P165: 0-8] [P166: 0-8] [P167: 0-8] [P168: 0-8] [P169: 0-8] [P170: 0-8] [P171: 0-8] [P172: 0-8] [P173: 0-8] [P174: 0-8] [P175: 0-8] [P176: 0-8] [P177: 0-8] [P178: 0-8] [P179: 0-8] [P180: 0-8] [P181: 0-8] [P182: 0-8] [P183: 0-8] [P184: 0-8] [P185: 0-8] [P186: 0-8] [P187: 0-8] [P188: 0-8] [P189: 0-8] [P190: 0-8] [P191: 0-8] [P192: 0-8] [P193: 0-8] [P194: 0-8] [P195: 0-8] [P196: 0-8] [P197: 0-8] [P198: 0-8] [P199: 0-8] [P200: 0-8] [P201: 0-8] [P202: 0-8] [P203: 0-8] [P204: 0-8] [P205: 0-8] [P206: 0-8] [P207: 0-8] [P208: 0-8] [P209: 0-8] [P210: 0-8] [P211: 0-8] [P212: 0-8] [P213: 0-8] [P214: 0-8] [P215: 0-8] [P216: 0-8] [P217: 0-8] [P218: 0-8] [P219: 0-8] [P220: 0-8] [P221: 0-8] [P222: 0-8] [P223: 0-8] [P224: 0-8] [P225: 0-8] [P226: 0-8] [P227: 0-8] [P228: 0-8] [P229: 0-8] [P230: 0-8] [P231: 0-8] [P232: 0-8] [P233: 0-8] [P234: 0-8] [P235: 0-8] [P236: 0-8] [P237: 0-8] [P238: 0-8] [P239: 0-8] [P240: 0-8] [P241: 0-8] [P242: 0-8] [P243: 0-8] [P244: 0-8] [P245: 0-8] [P246: 0-8] [P247: 0-8] [P248: 0-8] [P249: 0-8] [P250: 0-8] [P251: 0-8] [P252: 0-8] [P253: 0-8] [P254: 0-8] [P255: 0-8] [P256: 0-8] [P257: 0-8] [P258: 0-8] [P259: 0-8] [P260: 0-8] [P261: 0-8] [P262: 0-8] [P263: 0-8] [P264: 0-8] [P265: 0-8] [P266: 0-8] [P267: 0-8] [P268: 0-8] [P269: 0-8] [P270: 0-8] [P271: 0-8] [P272: 0-8] [P273: 0-8] [P274: 0-8] [P275: 0-8] [P276: 0-8] [P277: 0-8] [P278: 0-8] [P279: 0-8] [P280: 0-8] [P281: 0-8] [P282: 0-8] [P283: 0-8] [P284: 0-8] [P285: 0-8] [P286: 0-8] [P287: 0-8] [P288: 0-8] [P289: 0-8] [P290: 0-8] [P291: 0-8] [P292: 0-8] [P293: 0-8] [P294: 0-8] [P295: 0-8] [P296: 0-8] [P297: 0-8] [P298: 0-8] [P299: 0-8] [P300: 0-8] [P301: 0-8] [P302: 0-8] [P303: 0-8] [P304: 0-8] [P305: 0-8] [P306: 0-8] [P307: 0-8] [P308: 0-8] [P309: 0-8] [P310: 0-8] [P311: 0-8] [P312: 0-8] [P313: 0-8] [P314: 0-8] [P315: 0-8] [P316: 0-8] [P317: 0-8] [P318: 0-8] [P319: 0-8] [P320: 0-8] [P321: 0-8] [P322: 0-8] [P323: 0-8] [P324: 0-8] [P325: 0-8] [P326: 0-8] [P327: 0-8] [P328: 0-8] [P329: 0-8] [P330: 0-8] [P331: 0-8] [P332: 0-8] [P333: 0-8] [P334: 0-8] [P335: 0-8] [P336: 0-8] [P337: 0-8] [P338: 0-8] [P339: 0-8] [P340: 0-8] [P341: 0-8] [P342: 0-8] [P343: 0-8] [P344: 0-8] [P345: 0-8] [P346: 0-8] [P347: 0-8] [P348: 0-8] [P349: 0-8] [P350: 0-8] [P351: 0-8] [P352: 0-8] [P353: 0-8] [P354: 0-8] [P355: 0-8] [P356: 0-8] [P357: 0-8] [P358: 0-8] [P359: 0-8] [P360: 0-8] [P361: 0-8] [P362: 0-8] [P363: 0-8] [P364: 0-8] [P365: 0-8] [P366: 0-8] [P367: 0-8] [P368: 0-8] [P369: 0-8] [P370: 0-8] [P371: 0-8] [P372: 0-8] [P373: 0-8] [P374: 0-8] [P375: 0-8] [P376: 0-8] [P377: 0-8] [P378: 0-8] [P379: 0-8] [P380: 0-8] [P381: 0-8] [P382: 0-8] [P383: 0-8] [P384: 0-8] [P385: 0-8] [P386: 0-8] [P387: 0-8] [P388: 0-8] [P389: 0-8] [P390: 0-8] [P391: 0-8] [P392: 0-8] [P393: 0-8] [P394: 0-8] [P395: 0-8] [P396: 0-8] [P397: 0-8] [P398: 0-8] [P399: 0-8] [P400: 0-8] [P401: 0-8] [P402: 0-8] [P403: 0-8] [P404: 0-8] [P405: 0-8] [P406: 0-8] [P407: 0-8] [P408: 0-8] [P409: 0-8] [P410: 0-8] [P411: 0-8] [P412: 0-8] [P413: 0-8] [P414: 0-8] [P415: 0-8] [P416: 0-8] [P417: 0-8] [P418: 0-8] [P419: 0-8] [P420: 0-8] [P421: 0-8] [P422: 0-8] [P423: 0-8] [P424: 0-8] [P425: 0-8] [P426: 0-8] [P427: 0-8] [P428: 0-8] [P429: 0-8] [P430: 0-8] [P431: 0-8] [P432: 0-8] [P433: 0-8] [P434: 0-8] [P435: 0-8] [P436: 0-8] [P437: 0-8] [P438: 0-8] [P439: 0-8] [P440: 0-8] [P441: 0-8] [P442: 0-8] [P443: 0-8] [P444: 0-8] [P445: 0-8] [P446: 0-8] [P447: 0-8] [P448: 0-8] [P449: 0-8] [P450: 0-8] [P451: 0-8] [P452: 0-8] [P453: 0-8] [P454: 0-8] [P455: 0-8] [P456: 0-8] [P457: 0-8] [P458: 0-8] [P459: 0-8] [P460: 0-8] [P461: 0-8] [P462: 0-8] [P463: 0-8] [P464: 0-8] [P465: 0-8] [P466: 0-8] [P467: 0-8] [P468: 0-8] [P469: 0-8] [P470: 0-8] [P471: 0-8] [P472: 0-8] [P473: 0-8] [P474: 0-8] [P475: 0-8] [P476: 0-8] [P477: 0-8] [P478: 0-8] [P479: 0-8] [P480: 0-8] [P481: 0-8] [P482: 0-8] [P483: 0-8] [P484: 0-8] [P485: 0-8] [P486: 0-8] [P487: 0-8] [P488: 0-8] [P489: 0-8] [P490: 0-8] [P491: 0-8] [P492: 0-8] [P493: 0-8] [P494: 0-8] [P495: 0-8] [P496: 0-8] [P497: 0-8] [P498: 0-8] [P499: 0-8] [P500: 0-8] [P501: 0-8] [P502: 0-8] [P503: 0-8] [P504: 0-8] [P505: 0-8] [P506: 0-8] [P507: 0-8] [P508: 0-8] [P509: 0-8] [P510: 0-8] [P511: 0-8] [P512: 0-8] [P513: 0-8] [P514: 0-8] [P515: 0-8] [P516: 0-8] [P517: 0-8] [P518: 0-8] [P519: 0-8] [P520: 0-8] [P521: 0-8] [P522: 0-8] [P523: 0-8] [P524: 0-8] [P525: 0-8] [P526: 0-8] [P527: 0-8] [P528: 0-8] [P529: 0-8] [P530: 0-8] [P531: 0-8] [P532: 0-8] [P533: 0-8] [P534: 0-8] [P535: 0-8] [P536: 0-8] [P537: 0-8] [P538: 0-8] [P539: 0-8] [P540: 0-8] [P541: 0-8] [P542: 0-8] [P543: 0-8] [P544: 0-8] [P545: 0-8] [P546: 0-8] [P547: 0-8] [P548: 0-8] [P549: 0-8] [P550: 0-8] [P551: 0-8] [P552: 0-8] [P553: 0-8] [P554: 0-8] [P555: 0-8] [P556: 0-8] [P557: 0-8] [P558: 0-8] [P559: 0-8] [P560: 0-8] [P561: 0-8] [P562: 0-8] [P563: 0-8] [P564: 0-8] [P565: 0-8] [P566: 0-8] [P567: 0-8] [P568: 0-8] [P569: 0-8] [P570: 0-8] [P571: 0-8] [P572: 0-8] [P573: 0-8] [P574: 0-8] [P575: 0-8] [P576: 0-8] [P577: 0-8] [P578: 0-8] [P579: 0-8] [P580: 0-8] [P581: 0-8] [P582: 0-8] [P583: 0-8] [P584: 0-8] [P585: 0-8] [P586: 0-8] [P587: 0-8] [P588: 0-8] [P589: 0-8] [P590: 0-8] [P591: 0-8] [P592: 0-8] [P593: 0-8] [P594: 0-8] [P595: 0-8] [P596: 0-8] [P597: 0-8] [P598: 0-8] [P599: 0-8] [P600: 0-8] [P601: 0-8] [P602: 0-8] [P603: 0-8] [P604: 0-8] [P605: 0-8] [P606: 0-8] [P607: 0-8] [P608: 0-8] [P609: 0-8] [P610: 0-8] [P611: 0-8] [P612: 0-8] [P613: 0-8] [P614: 0-8] [P615: 0-8] [P616: 0-8] [P617: 0-8] [P618: 0-8] [P619: 0-8] [P620: 0-8] [P621: 0-8] [P622: 0-8] [P623: 0-8] [P624: 0-8] [P625: 0-8] [P626: 0-8] [P627: 0-8] [P628: 0-8] [P629: 0-8] [P630: 0-8] [P631: 0-8] [P632: 0-8] [P633: 0-8] [P634: 0-8] [P635: 0-8] [P636: 0-8] [P637: 0-8] [P638: 0-8] [P639: 0-8] [P640: 0-8] [P641: 0-8] [P642: 0-8] [P643: 0-8] [P644: 0-8] [P645: 0-8] [P646: 0-8] [P647: 0-8] [P648: 0-8] [P649: 0-8] [P650: 0-8] [P651: 0-8] [P652: 0-8] [P653: 0-8] [P654: 0-8] [P655: 0-8] [P656: 0-8] [P657: 0-8] [P658: 0-8] [P659: 0-8] [P660: 0-8] [P661: 0-8] [P662: 0-8] [P663: 0-8] [P664: 0-8] [P665: 0-8] [P666: 0-8] [P667: 0-8] [P668: 0-8] [P669: 0-8] [P670: 0-8] [P671: 0-8] [P672: 0-8] [P673: 0-8] [P674: 0-8] [P675: 0-8] [P676: 0-8] [P677: 0-8] [P678: 0-8] [P679: 0-8] [P680: 0-8] [P681: 0-8] [P682: 0-8] [P683: 0-8] [P684: 0-8] [P685: 0-8] [P686: 0-8] [P687: 0-8] [P688: 0-8] [P689: 0-8] [P690: 0-8] [P691: 0-8] [P692: 0-8] [P693: 0-8] [P694: 0-8] [P695: 0-8] [P696: 0-8] [P697: 0-8] [P698: 0-8] [P699: 0-8] [P700: 0-8] [P701: 0-8] [P702: 0-8] [P703: 0-8] [P704: 0-8] [P705: 0-8] [P706: 0-8] [P707: 0-8] [P708: 0-8] [P709: 0-8] [P710: 0-8] [P711: 0-8] [P712: 0-8] [P713: 0-8] [P714: 0-8] [P715: 0-8] [P716: 0-8] [P717: 0-8] [P718: 0-8] [P719: 0-8] [P720: 0-8] [P721: 0-8] [P722: 0-8] [P723: 0-8] [P724: 0-8] [P725: 0-8] [P726: 0-8] [P727: 0-8] [P728: 0-8] [P729: 0-8] [P730: 0-8] [P731: 0-8] [P732: 0-8] [P733: 0-8] [P734: 0-8] [P735: 0-8] [P736: 0-8] [P737: 0-8] [P738: 0-8] [P739: 0-8] [P740: 0-8] [P741: 0-8] [P742: 0-8] [P743: 0-8] [P744: 0-8] [P745: 0-8] [P746: 0-8] [P747: 0-8] [P748: 0-8] [P749: 0-8] [P750: 0-8] [P751: 0-8] [P752: 0-8] [P753: 0-8] [P754: 0-8] [P755: 0-8] [P756: 0-8] [P757: 0-8] [P758: 0-8] [P759: 0-8] [P760: 0-8] [P761: 0-8] [P762: 0-8] [P763: 0-8] [P764: 0-8] [P765: 0-8] [P766: 0-8] [P767: 0-8] [P768: 0-8] [P769: 0-8] [P770: 0-8] [P771: 0-8] [P772: 0-8] [P773: 0-8] [P774: 0-8] [P775: 0-8] [P776: 0-8] [P777: 0-8] [P778: 0-8] [P779: 0-8] [P780: 0-8] [P781: 0-8] [P782: 0-8] [P783: 0-8] [P784: 0-8] [P785: 0-8] [P786: 0-8] [P787: 0-8] [P788: 0-8] [P789: 0-8] [P790: 0-8] [P791: 0-8] [P792: 0-8] [P793: 0-8] [P794: 0-8] [P795: 0-8] [P796: 0-8] [P797: 0-8] [P798: 0-8] [P799: 0-8] [P800: 0-8] [P801: 0-8] [P802: 0-8] [P803: 0-8] [P804: 0-8] [P805: 0-8] [P806: 0-8] [P807: 0-8] [P808: 0-8] [P809: 0-8] [P810: 0-8] [P811: 0-8] [P812: 0-8] [P813: 0-8] [P814: 0-8] [P815: 0-8] [P816: 0-8] [P817: 0-8] [P818: 0-8] [P819: 0-8] [P820: 0-8] [P821: 0-8] [P822: 0-8] [P823: 0-8] [P824: 0-8] [P825: 0-8] [P826: 0-8] [P827: 0-8] [P828: 0-8] [P829: 0-8] [P830: 0-8] [P831: 0-8] [P832: 0-8] [P833: 0-8] [P834: 0-8] [P835: 0-8] [P836: 0-8] [P837: 0-8] [P838: 0-8] [P839: 0-8] [P840: 0-8] [P841: 0-8] [P842: 0-8] [P843: 0-8] [P844: 0-8] [P845: 0-8] [P846: 0-8] [P847: 0-8] [P848: 0-8] [P849: 0-8] [P850: 0-8] [P851: 0-8] [P852: 0-8] [P853: 0-8] [P854: 0-8] [P855: 0-8] [P856: 0-8] [P857: 0-8] [P858: 0-8] [P859: 0-8] [P860: 0-8] [P861: 0-8] [P862: 0-8] [P863: 0-8] [P864: 0-8] [P865: 0-8] [P866: 0-8] [P867: 0-8] [P868: 0-8] [P869: 0-8] [P870: 0-8] [P871: 0-8] [P872: 0-8] [P873: 0-8] [P874: 0-8] [P875: 0-8] [P876: 0-8] [P877: 0-8] [P878: 0-8] [P879: 0-8] [P880: 0-8] [P881: 0-8] [P882: 0-8] [P883: 0-8] [P884: 0-8] [P885: 0-8] [P886: 0-8] [P887: 0-8] [P888: 0-8] [P889: 0-8] [P890: 0-8] [P891: 0-8] [P892: 0-8] [P893: 0-8] [P894: 0-8] [P895: 0-8] [P896: 0-8] [P897: 0-8] [P898: 0-8] [P899: 0-8] [P900: 0-8] [P901: 0-8] [P902: 0-8] [P903: 0-8] [P904: 0-8] [P905: 0-8] [P906: 0-8] [P907: 0-8] [P908: 0-8] [P909: 0-8] [P910: 0-8] [P911: 0-8] [P912: 0-8] [P913: 0-8] [P914: 0-8] [P915: 0-8] [P916: 0-8] [P917: 0-8] [P918: 0-8] [P919: 0-8] [P920: 0-8] [P921: 0-8] [P922: 0-8] [P923: 0-8] [P924: 0-8] [P925: 0-8] [P926: 0-8] [P927: 0-8] [P928: 0-8] [P929: 0-8] [P930: 0-8] [P931: 0-8] [P932: 0-8] [P933: 0-8] [P934: 0-8] [P935: 0-8] [P936: 0-8] [P937: 0-8] [P938: 0-8] [P939: 0-8] [P940: 0-8] [P941: 0-8] [P942: 0-8] [P943: 0-8] [P944: 0-8] [P945: 0-8] [P946: 0-8] [P947: 0-8] [P948: 0-8] [P949: 0-8] [P950: 0-8] [P951: 0-8] [P952: 0-8] [P953: 0-8] [P954: 0-8] [P955: 0-8] [P956: 0-8] [P957: 0-8] [P958: 0-8] [P959: 0-8] [P960: 0-8] [P961: 0-8] [P962: 0-8] [P963: 0-8] [P964: 0-8] [P965: 0-8] [P966: 0-8] [P967: 0-8] [P968: 0-8] [P969: 0-8] [P970: 0-8] [P971: 0-8] [P972: 0-8] [P973: 0-8] [P974: 0-8] [P975: 0-8] [P976: 0-8] [P977: 0-8] [P978: 0-8] [P979: 0-8] [P980: 0-8] [P981: 0-8] [P982: 0-8] [P983: 0-8] [P984: 0-8] [P985: 0-8] [P986: 0-8] [P987: 0-8] [P988: 0-8] [P989: 0-8] [P990: 0-8] [P991: 0-8] [P992: 0-8] [P993: 0-8] [P994: 0-8] [P995: 0-8] [P996: 0-8] [P997: 0-8] [P998: 0-8] [P999: 0-8] [P1000: 0-8] [P1001: 0-8] [P1002: 0-8] [P1003: 0-8] [P1004: 0-8] [P1005: 0-8] [P1006: 0-8] [P1007: 0-8] [P1008: 0-8] [P1009: 0-8]
```

OUTPUT 17:

```
D:\OS lab3\Task17.exe
Timeline: [P1: 0-1] [P2: 1-5] [P1: 5-10] [P3: 10-12]
PID | Completion | Waiting Time
P1 | 12 | 6
P2 | 7 | 2
P3 | 4 | 0

Process exited after 0.3583 seconds with return value 0
Press any key to continue . . .
```

CODE 18:

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 struct Process {
6     string id;
7     int arrival_time;
8     int burst_time;
9     int remaining_time;
10    int completion_time;
11    int turnaround_time;
12    int wait_time;
13    bool in_queue = false;
14 };
15
16 void schedule_roundRobin(vector<Process>& processes, int quantum) {
17     int queue_idx = 0, completed = 0, n = processes.size();
18     int current_time = 0;
19     for (int i = 0; i < n; i++) {
20         if (processes[i].arrival_time == 0) {
21             queue_idx = i;
22             processes[i].in_queue = true;
23         }
24     }
25     string queue = "Queue = ";
26     while (!queue.empty()) {
27         int id = queue.front();
28         queue.pop();
29         Process p = processes[id];
30         int slice = min(quantum, p.remaining_time);
31         p.remaining_time -= slice;
32         current_time += slice;
33         if (p.remaining_time == 0) {
34             cout << "P" << p.id << " completed at " << current_time << endl;
35             completed++;
36             if (completed == n) break;
37         }
38         for (int i = 0; i < n; i++) {
39             if (i != id && processes[i].arrival_time <= current_time && processes[i].remaining_time > 0 && !processes[i].in_queue) {
40                 queue.push(i);
41                 processes[i].in_queue = true;
42             }
43         }
44         if (p.remaining_time > 0) {
45             queue.push(id);
46         }
47         p.completion_time = current_time;
48         p.turnaround_time = p.completion_time - p.arrival_time;
49         p.wait_time = p.turnaround_time - p.burst_time;
50         p.in_queue = false;
51     }
52     cout << "P" << p.id << " turnaround | wait\n";
53     for (const auto& p : processes) {
54         cout << p.id << " | " << p.turnaround_time << " | " << p.wait_time << " | \n";
55     }
56 }
57
58 int main() {
59     vector<Process> processes = {
60         { "P1", 0, 1, 1, 0, 0, 0 },
61         { "P2", 1, 5, 5, 0, 0, 0 },
62         { "P3", 10, 2, 2, 0, 0, 0 }
63     };
64     schedule_roundRobin(processes, 2);
65 }
```

OUTPUT 18:

```
D:\OS lab3\Task18.exe
Quantum = 2
Execution : P1 (2 ms ) -> P2 (2 ms ) -> P3 (2 ms ) -> P1 (1 ms ) -> P3 (2 ms )
PID | Turnaround | Wait
P1 | 7 | 4
P2 | 4 | 2
P3 | 9 | 5

Process exited after 0.3802 seconds with return value 0
Press any key to continue . . .
```

CODE 19:

```
1 #include <iostream>
2 #include <vector>
3
4 struct Process {
5     int id;
6     int priority;
7     int burst_time;
8     int arrival_time;
9     int remaining_time;
10    int completion_time;
11    int wait_time;
12 };
13
14 void schedule_preemptive_priority(std::vector<Process> & processes) {
15     int current_time = 0, completed = 0, n = processes.size();
16     while (completed < n) {
17         int min_id = -1;
18         int highest_priority = INT_MIN;
19         for (int i = 0; i < n; i++) {
20             if (processes[i].arrival_time <= current_time && processes[i].remaining_time > 0) {
21                 if (processes[i].priority < highest_priority) {
22                     highest_priority = processes[i].priority;
23                     min_id = i;
24                 }
25             }
26         }
27         if (min_id == -1) {
28             current_time++;
29             continue;
30         }
31         processes[min_id].remaining_time--;
32         current_time++;
33         if (processes[min_id].remaining_time == 0) {
34             processes[min_id].completion_time = current_time;
35             processes[min_id].wait_time = (processes[min_id].completion_time - processes[min_id].arrival_time) - processes[min_id].burst_time;
36             completed++;
37         }
38     }
39     std::cout << "PID | Priority | Burst | Arrival | Waiting Time\n";
40     for (const auto & p : processes) {
41         std::cout << p.id << " | " << p.priority << " | " << p.burst_time
42             << " | " << p.arrival_time << " | " << p.wait_time << "\n";
43     }
44 }
45
46 int main() {
47     std::vector<Process> processes = {
48         {1, 2, 4, 0, 3},
49         {2, 1, 3, 1, 0},
50         {3, 3, 2, 2, 5}
51     };
52     schedule_preemptive_priority(processes);
53 }
```

OUTPUT 19:

```
D:\OS lab3\Task.19.exe
PID | Priority | Burst | Arrival | Waiting Time
P1 | 2 | 4 | 0 | 3
P2 | 1 | 3 | 1 | 0
P3 | 3 | 2 | 2 | 5

-----
Process exited after 0.3476 seconds with return value 0
Press any key to continue . . .
```

CODE 20:

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Time 0: P1 enters Q1 ( runs 2 ms ) -> Demoted to Q2" << std::endl;
5     std::cout << "Time 2: P2 enters Q1 ( runs 1 ms , finishes ) -> Terminated" << std::endl;
6     std::cout << "Time 3: P1 from Q2 runs (4 ms ) -> Demoted to Q3" << std::endl;
7     std::cout << "Time 7: Aging trigger -> P1 promoted back to Q1" << std::endl;
8     std::cout << "Time 7: P1 finishes in Q1 . All jobs completed ." << std::endl;
9     return 0;
10 }
```

OUTPUT 20:

```
D:\OS lab3\Task.20.exe
Time 0: P1 enters Q1 ( runs 2 ms ) -> Demoted to Q2
Time 2: P2 enters Q1 ( runs 1 ms , finishes ) -> Terminated
Time 3: P1 from Q2 runs (4 ms ) -> Demoted to Q3
Time 7: Aging trigger -> P1 promoted back to Q1
Time 7: P1 finishes in Q1 . All jobs completed .

-----
Process exited after 0.4105 seconds with return value 0
Press any key to continue . . .
```
